

Cerințe obligatorii

1. Pattern-urile implementate trebuie sa respecte definiția din GoF discutată în cadrul cursurilor și laboratoarelor. Nu sunt acceptate variații sau implementări incomplete.
2. Pattern-ul trebuie implementat în totalitate corect pentru a fi luat în calcul.
3. Soluția nu conține erori de compilare.
4. Pattern-urile pot fi tratate distinct sau pot fi implementate pe același set de clase.
5. Implementările care nu au legătura funcțională cu cerințele din subiect NU vor fi luate în calcul (preluare unui exemplu din alte surse nu va fi punctată).
6. NU este permisă modificare claselor primite.
7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

Cerințe Clean Code obligatorii (soluția este depunctata cu câte 2 puncte pentru fiecare cerința ce nu este respectată) - maxim se pot pierde 8 puncte

1. Pentru denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
2. Pattern-urile și clasa ce conține metoda main() sunt definite in pachete distincte ce au forma *cts.nume.prenume.gNrGrupa.denumire_pattern*, *cts.nume.prenume.gNrGrupa.main* (studenții din anul suplimentar trec "as" în loc de gNrGrupa);
3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie sa aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care sa simuleze acțiunea cerută sau vor implementa prelucrări simple.

Se dezvoltă o aplicație software destinată unui restaurant.

- 5p.** Realizati o aplicatie care are rolul de a stoca structura unui pachet format din urmatoarele componente: interfete, clase, metode, functii. O functionalitate a acestei aplicatii este de a analiza pachetul printr-o parsare recursiva a codului. Integrati un design pattern ce permite aceasta functionalitate de analiza a tuturor elementelor de tip Item.
- 1p.** Sa se motiveze in scris alegerea design pattern-ului care modeleaza cel mai bine situatia enuntata. Motivarea trebuie sa contina elementele cheie care denota utilizarea respectivului design pattern.
- 3p.** Să se testeze soluția prin crearea unui pachet format din clase, interfete si metode si cererea spre analiza a unei componente de tip clasa precum si metoda.
- 5p.** In cadrul aplicatiei mai sus mentionata, se doreste adaugarea unei functionalitati suplimentare de afisare a unui cod. Astfel, atunci cand se afiseaza un cod, se doreste adaugarea automata folosind un AI tool a rezumatului codului aferent. Implementarea trebuie sa permita adaugarea de alte functionalitati in timp.
- 1p.** Sa se motiveze in scris alegerea design pattern-ului care modeleaza cel mai bine situatia enuntata. Motivarea trebuie sa contina elementele cheie care denota utilizarea respectivului design pattern.
- 3p.** Să se testeze soluția prin afisarea unei obiect de tip Clasa folosind rezumatul precum si pe baza versiunii anterioare de afisare.